



Laboratorio No.1 – Semana 14 - Integración servicios en la nube

Nombre: Angel Julian Garcia Lopez **Carné:**1097925

Objetivo: Desarrollar y publicar una aplicación web que permita interactuar con archivos almacenados en Azure Blob Storage, utilizando control de versiones con Git y un pipeline (script) CI/CD de Azure DevOps para automatizar el despliegue de la aplicación en Azure App Service

Resultado esperado

- Aplicación web funcional ejecutándose en Azure App Service.
- Conexión entre App Service y Azure Blob Storage.
- Listado y consulta de archivos almacenados en un contenedor Blob.
- Operación de carga de archivos al Blob Storage.
- Repositorio Git con historial de cambios.
- Pipeline de Azure DevOps que compile y despliegue automáticamente la aplicación.
- Evidencias que demuestren el funcionamiento de la solución.

Repositorio recomendado

Utilice como punto de partida el ejemplo oficial de Microsoft/Azure Samples revisado para esta actividad:

GitHub: <https://github.com/Azure-Samples/serviceconnector-webapp-storageblob-dotnet>

El tutorial oficial utiliza una aplicación web .NET conectada a Azure Blob Storage mediante una identidad administrada y Service Connector. La documentación indica además que la aplicación puede desplegarse en Azure App Service.

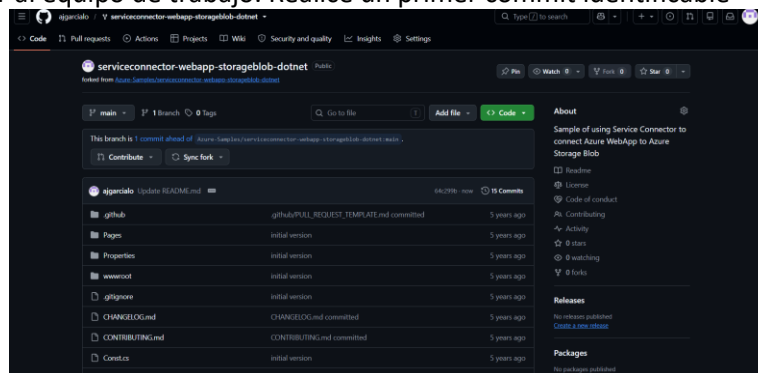
Tecnologías

Tecnología	Uso
C# / .NET	Desarrollo de la aplicación web.
Azure Blob Storage	Almacenamiento de archivos.
Azure App Service	Hospedaje de la aplicación.
Git	Control de versiones.
Azure DevOps Pipelines	Integración y despliegue continuo.
Managed Identity / Service Connector	Acceso seguro desde App Service al almacenamiento.

Realización de la actividad

1. Preparar el repositorio

En el repositorio brindado arriba, realice un fork del repositorio base o copie el proyecto a un repositorio propio. El repositorio debe pertenecer al equipo de trabajo. Realice un primer commit identificable

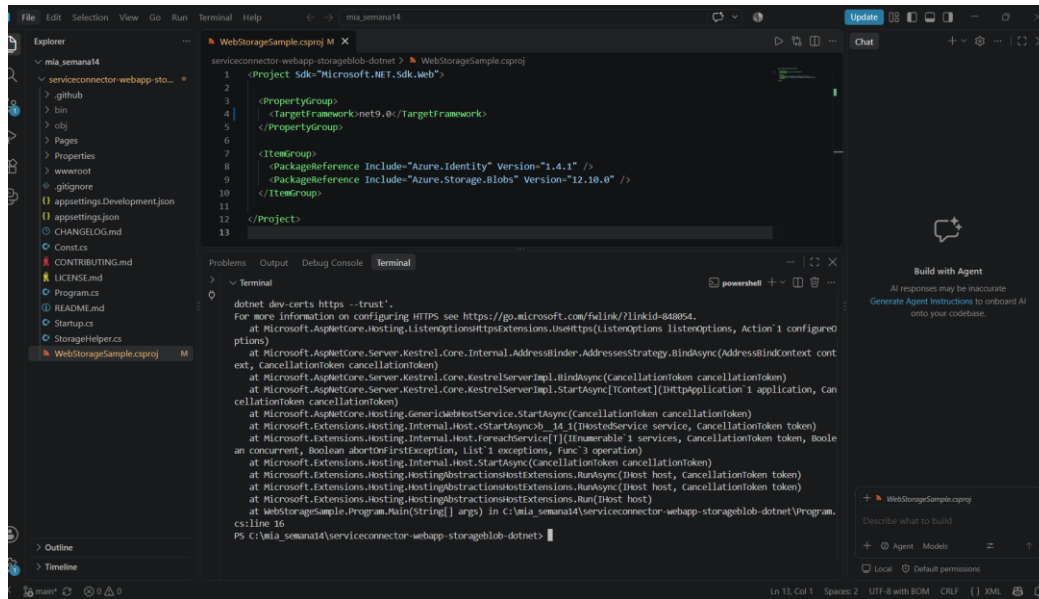




2. Revisar la aplicación:

Ejecute la aplicación localmente y revise cómo obtiene acceso al Blob Storage. Identifique el punto del código donde se realiza la lectura/escritura de blobs y documente brevemente su funcionamiento. Visual Studio -> carpeta -> MIA_Semana14 -> git clone [repo url] -> dotnet commands

NOTA: cambia ren .csproj file -> <TargetFramework>net9.0</TargetFramework>



3. Crear DotNet Env

- En terminal, instalar: **dotnet add package DotNetEnv**
- Crear .env file en raíz de proyecto, valores ->
 AZURE_STORAGEBLOB_RESOURCEENDPOINT=https://miarchivos2026.blob.core.windows.net/
 AZURE_STORAGE_CONTAINER_NAME=default
- Crear .env.production con los mismos valores en el mismo lugar
- Anadir cambio brindado en program.cs
- dotnet clean -> dotnet restore -> dotnet build -> dotnet run



```

1  using DotNetEnv;
2
3  using Microsoft.AspNetCore.Hosting;
4
5  using Microsoft.Extensions.Hosting;
6
7  namespace WebStorageSample
8  {
9
10     {
11         0 references
12         public class Program
13
14     {
15         0 references
16         public static void Main(string[] args)
17         {
18             Env.Load(".env");
19             CreateHostBuilder(args).Build().Run();
20         }
21     }
22 }
23
24

```

Problems | Output | Debug Console | Terminal

Terminal

```

C:\mia_semana14\serviceconnector-webapp-storageblob-dotnet\StorageHelper.cs(34,17): warning CA2280: Si inicia de nuev
o la excepción detectada, se cambia la información de la pila. (https://learn.microsoft.com/dotnet/fundamentals/code-an
alysis/quality-rules/ca2280)
C:\mia_semana14\serviceconnector-webapp-storageblob-dotnet\StorageHelper.cs(66,17): warning CA2280: Si inicia de nuev
o la excepción detectada, se cambia la información de la pila. (https://learn.microsoft.com/dotnet/fundamentals/code-an
alysis/quality-rules/ca2280)

Compilación correcto con 10 advertencias en 4.0s
PS C:\mia_semana14\serviceconnector-webapp-storageblob-dotnet>

```

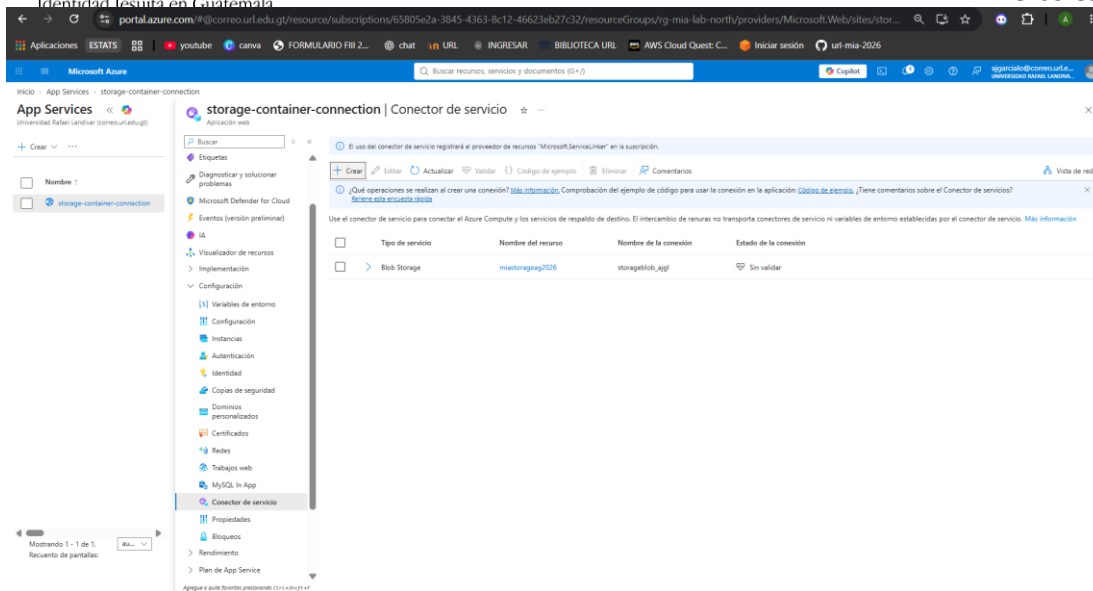
4. Crear recursos en Azure

En azure portal site, cree o utilice un Resource Group, una Storage Account con Blob Storage y un contenedor. El contenedor deberá tener al menos tres archivos de prueba, Puede utilizar el creado en semana 13.

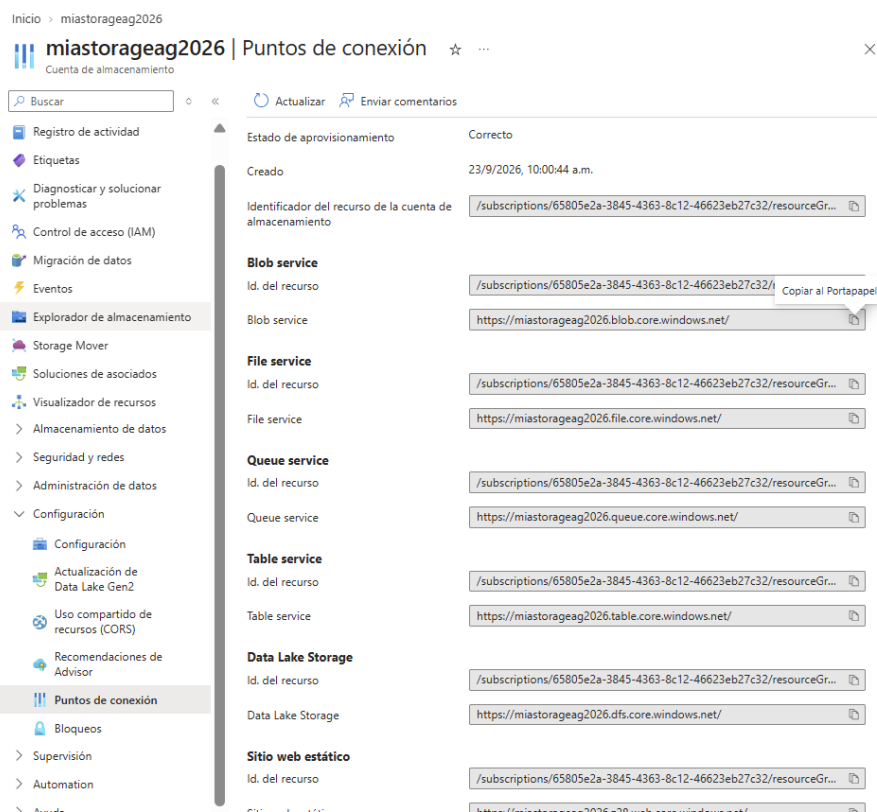
5. Conectar App Service con Blob Storage

Configure la conexión entre App Service y Blob Storage mediante Service Connector utilizando identidad administrada. Verifique que la identidad tenga los permisos necesarios sobre los blobs.

- Azure Portal → **Storage accounts** → tu Storage Account → Data storage → Containers
- Crear contenedor "default"
- Activa identidad administrada en App Service: **Azure Portal** → App Service → Settings → Identity → System assigned: on → Save (web app creada semana 13)
- Configurar la conexión entre App Service y Blob Storage: **App Service** → Settings → Service Connector → Create → Seleccionar "Storage Blob" → Seleccionar Storage Account [tu_blob_name] → Authentication "System-assigned managed identity" → Review + Create.



- e) Reemplazar storage blob endpoint y container nombre en .env file
Storage Account → Settings → Endpoints → Blob service



6. Probar el acceso al almacenamiento:

Ejecute la aplicación local y compruebe que puede leer los blobs. Realice al menos una operación de carga de archivo y verifique desde Azure Portal que el archivo llegó al contenedor. Visual Studio -> dotnet run

- Verifique que Azure CLI esté instalado ejecutando en la terminal de VS Code: `az --version`
- Inicie sesión en Azure: `az login`
- Verifique la cuenta de Azure activa: `az account show`
- Verifique que pueda obtener un token para Azure Storage: `az account get-access-token --resource https://storage.azure.com/`
- Si se muestra un accessToken la autenticación esta correcta, ejecutar -> `dotnet run`



f) Si hay error en acceso, ejecutar en terminal vs code: az role assignment create --assignee "TU_CORREO_DE_AZURE" --role "Storage Blob Data Contributor" --scope "/subscriptions/TU_SUBSCRIPTION_ID/resourceGroups/TU_RESOURCE_GROUP/providers/Microsoft.Storage/storageAccounts/TU_STORAGE"

Obtener info: az account show --query "{subscriptionId:id, user:user.name}" -o table
az storage account show --name "tu_storage" --query "{id:id, resourceGroup:resourceGroup}" -o table
g) Cerrar process ctr +c y logear de nuevo: az logout y az login, Por ultimo dotnet run

WebStorageSample Home Privacy

Hello Service Connector! UTC Now: 7/10/2026
23:54:47 +00:00.

[Learn about building Web apps with ASP.NET Core.](#)

Microsoft Azure

Inicio

default

Contenedor

Buscar

Agregar directorio Cargar Cambiar nivel de acceso Actualizar Eliminar Copiar Pegar Cambiar nombre Adquirir concesión Interrumpir concesión Editar columnas

Información general

default

Método de autenticación: Clave de acceso (Cambiar a la cuenta de usuario de Microsoft Entra)

Agregar filtro

Buscar blobs por prefijo (distingue mayúsculas de minúsculas)

Solo mostrar blobs activos

Mostrando todos los elementos 1

Nombre	Última modificación	Nivel de acceso	Tipo de blob	Tamaño	Estado de con...
HelloWorld	7/10/2026, 5:56:58 p.m.	Frecuente (inferido)	Blob en bloques	60 B	Disponible

Agregue o quite favoritos presionando Ctrl+Shift+F

Nota: no olvidar subir cambios a repo github fork



7. Crear Pipeline Yml

En la raíz de tu fork de GitHub crear archivo "azure-pipelines.yml"

Colocar en el archivo lo siguiente:

trigger:

- main

pool:

vmImage: 'windows-latest'

variables:

buildConfiguration: 'Release'

dotnetVersion: '9.x'

Completar posteriormente

azureSubscription: 'NOMBRE-DE-TU-SERVICE-CONNECTION'

appName: 'NOMBRE-DE-TU-APP-SERVICE'

steps:

1. Instalar .NET 9

- task: UseDotNet@2

displayName: 'Instalar .NET 9 SDK'

inputs:

packageType: 'sdk'

version: '\$(dotnetVersion)'

2. Restaurar dependencias

- task: DotNetCoreCLI@2

displayName: 'Restaurar dependencias'

inputs:

command: 'restore'

projects: '**/*.csproj'

3. Configurar ambiente de producción

- powershell: |

Copy-Item ".env.production" ".env" -Force

displayName: 'Configurar .env de producción'

4. Compilar

- task: DotNetCoreCLI@2

displayName: 'Compilar aplicación'

inputs:

command: 'build'

projects: '**/*.csproj'

arguments: '--configuration \$(buildConfiguration) --no-restore'



```
- task: DotNetCoreCLI@2
  displayName: 'Publicar aplicación'
  inputs:
    command: 'publish'
    projects: '**/*.csproj'
    arguments: '--configuration $(buildConfiguration) --output $(Build.ArtifactStagingDirectory)/publish --no-build'
  zipAfterPublish: true
```

6. Desplegar en Azure App Service

```
- task: AzureWebApp@1
  displayName: 'Desplegar en Azure App Service'
  inputs:
    azureSubscription: '$(azureSubscription)'
    appType: 'webApp'
    appName: '$(appName)'
    package: '$(Build.ArtifactStagingDirectory)/publish/**/*.zip'
```

```
github.com/ajgarcialo/serviceconnector-webapp-storageblob-dotnet/blob/main/azure-pipelines.yml
Aplicaciones ESTATS | youtube | canva | FORMULARIO FIII 2... | chat | URL | INGRESAR | BIBLIOTECA URL | AWS Cloud Quest: C... | Iniciar sesión | url-mia-2026

Files
main
Go to file
.github
Pages
Properties
wwwroot
.env.production
.gitignore
CHANGELOG.md
CONTRIBUTING.md
Const.cs
LICENSE.md
Program.cs
README.md
Startup.cs
StorageHelper.cs
WebStorageSample.csproj
appsettings.Development.json
appsettings.json
azure-pipelines.yml

serviceconnector-webapp-storageblob-dotnet / azure-pipelines.yml
Code Blame 60 lines (50 loc) · 1.43 KB
Raw [copy] [download] [edit] [refresh]

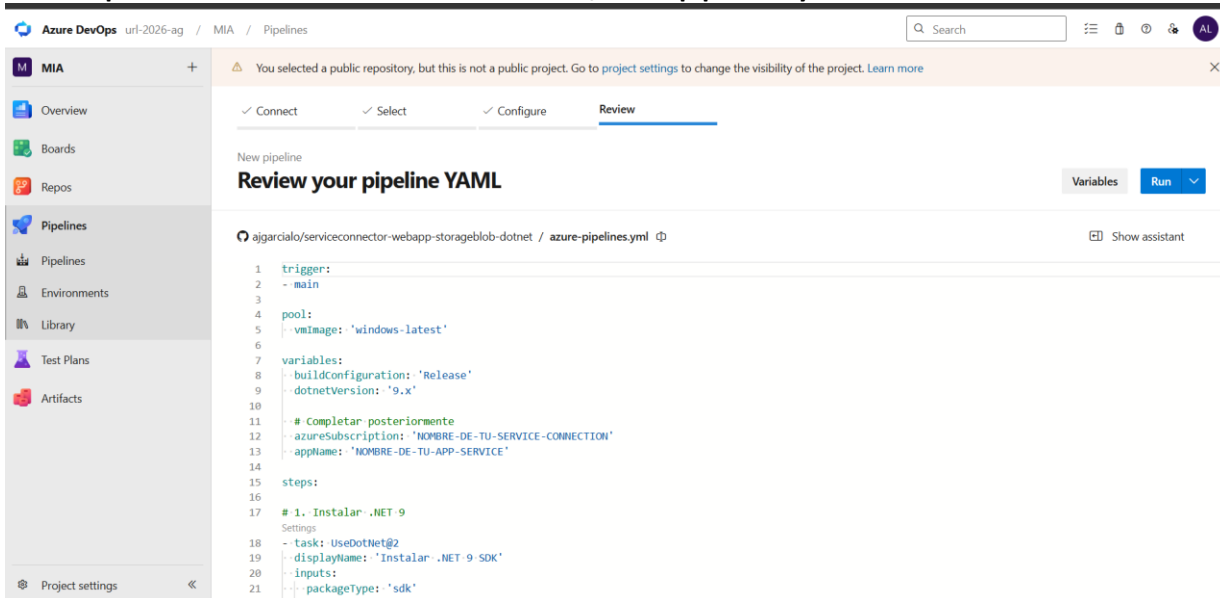
31 # 3. Configurar ambiente de producción
32 - powershell: |
33   Copy-Item ".env.production" ".env" -force
34   displayName: 'Configurar .env de producción'
35
36 # 4. Compilar
37 - task: DotNetCoreCLI@2
38   displayName: 'Compilar aplicación'
39   inputs:
40     command: 'build'
41     projects: '**/*.csproj'
42     arguments: '--configuration $(buildConfiguration) --no-restore'
43
44 # 5. Publicar
45 - task: DotNetCoreCLI@2
46   displayName: 'Publicar aplicación'
47   inputs:
48     command: 'publish'
49     projects: '**/*.csproj'
50     arguments: '--configuration $(buildConfiguration) --output $(Build.ArtifactStagingDirectory)/publish --no-build'
51     zipAfterPublish: true
52
53 # 6. Desplegar en Azure App Service
54 - task: AzureWebApp@1
55   displayName: 'Desplegar en Azure App Service'
56   inputs:
57     azureSubscription: '$(azureSubscription)'
58     appType: 'webApp'
59     appName: '$(appName)'
60     package: '$(Build.ArtifactStagingDirectory)/publish/**/*.zip'
```



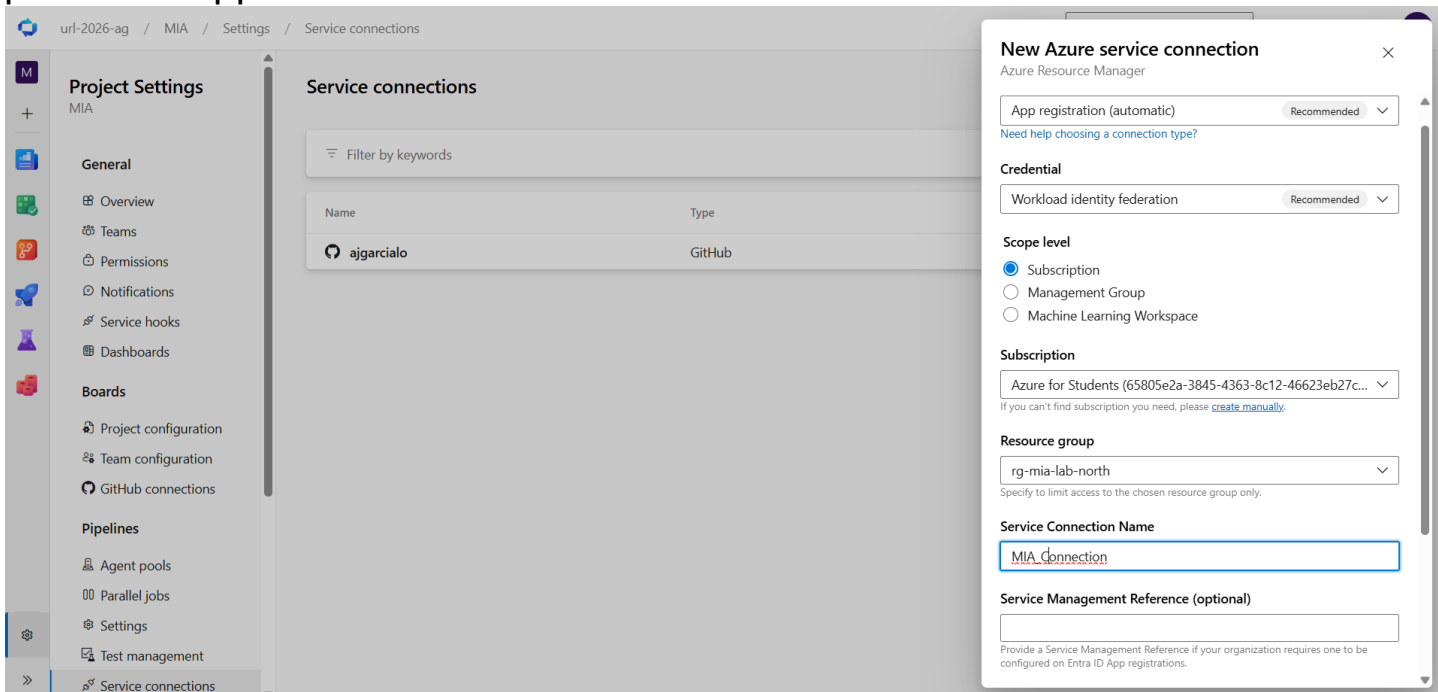
8. Crear pipeline CI/CD

Cree un pipeline YAML que restaure dependencias, compile, publique y despliegue la aplicación en Azure App Service.

- Conectarse a Azure DevOps son misma cuenta que azure portal -> <https://aex.dev.azure.com/me>
- Crear pipeline: En Azure devOps -> Menu izquierda -> pipelines -> Create Pipeline -> Seleccionar Github -> Autoriza Azure DevOps para acceder a GitHub -> seleccionar repositorio que hizo fork -> seleccionar Existing Azure Pipelines YAML file -> Branch: main Path: /azure-pipelines.yml



- Probar seleccionando "run"
- Conectar Azure DevOps con Azure Portal: Project Settings → Service connections → New service connection -> Azure Resource Manager -> luego, Workload identity federation -> Selecciona tu suscripción de Azure -> y resource group donde estan tus recursos -> Anadir nombre "MIA_Connection" -> Seleccionar "grant access permission to all pipelines" -> Save





9. Conectar Pipeline con App

- Obtener nombre app: En Azure Portal App Services → tu aplicación → Copia nombre app
- en github en tu "azure-pipelines.yml" file busca: azureSubscription y appName, cambiar por valores reales.

azureSubscription: TU_Service_Connection_name
appName: Tu_app_name

```
1 trigger:
2   - main
3
4 pool:
5   vmImage: 'windows-latest'
6
7 variables:
8   buildConfiguration: 'Release'
9   dotnetVersion: '9.x'
10
11 # Completar posteriormente
12 azureSubscription: 'MIA_Connection'
13 appName: 'storage-container-connection'
14
15 steps:
16
17 # 1. Instalar .NET 9
18 - task: UseDotNet@2
19   displayName: 'Instalar .NET 9 SDK'
20   inputs:
21     packageType: 'sdk'
22     version: '$(dotnetVersion)'
```

- Al hacer commit, verificar el pipeline se ejecuta automáticamente

Azure DevOps url-2026-ag / MIA / Pipelines / ajgarcialo.serviceconnector-...

ajgarcialo.serviceconnector-webapp-storageblob-dotnet [Edit] [Run pipeline]

Runs Stages Branches Analytics

Description	Stages	Time
#20261008.2 • feat(blob): pipeline yml change Individual CI for main b2bb7523	[Clock icon]	Just now
#20261008.1 • Update azure-pipelines.yml Manually run by main c4fe825d	[Checkmark icon]	5m ago 3m 24s



10. Validar la solución

Compruebe desde la URL pública que la aplicación funciona y que el acceso al Blob Storage continúa operativo después del despliegue.

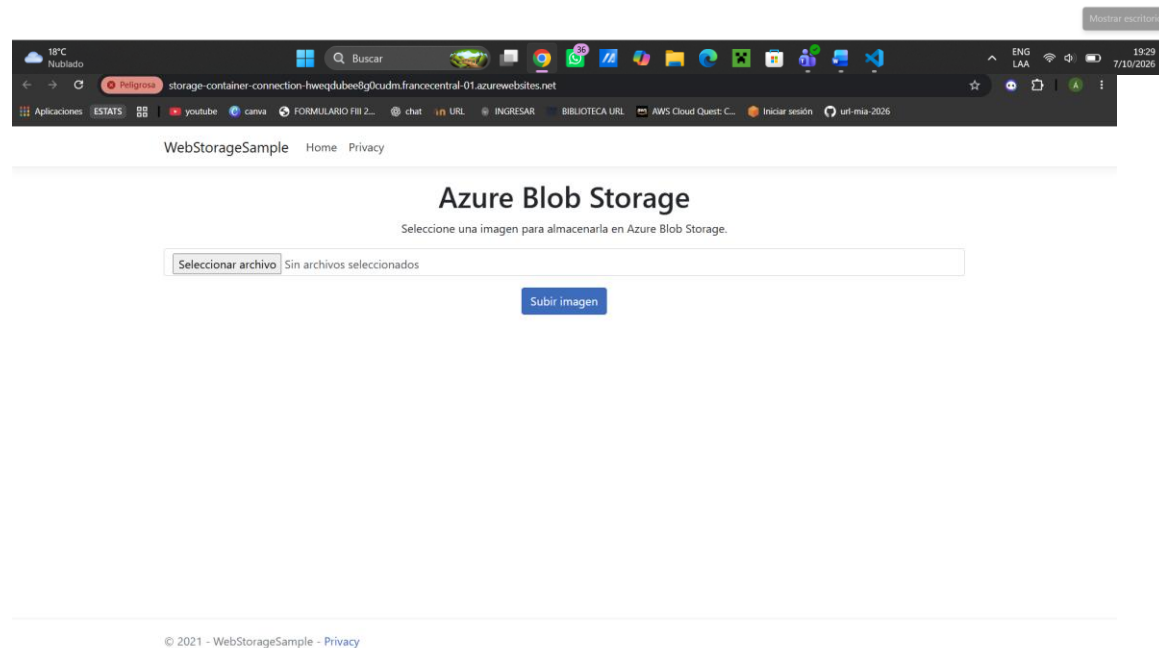
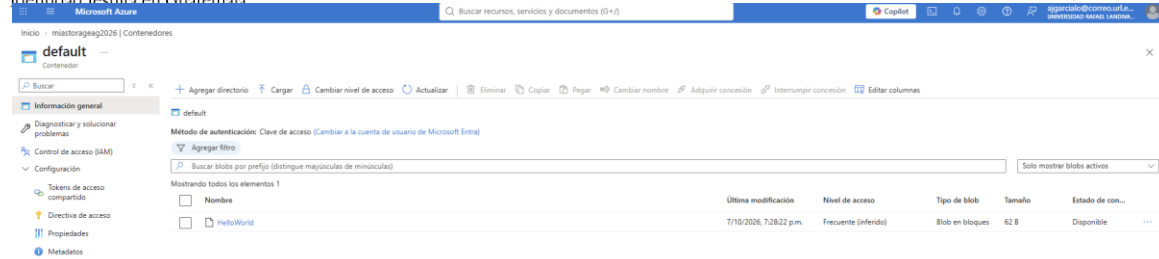
- a) acceso de la aplicación al Blob: Azure Portal → App Service → Settings → Environment variables → App settings
-> colocar AZURE_STORAGEBLOB_RESOURCEENDPOINT

https://tu_storage.blob.core.windows.net/

(si existe obviar)

- b) Verificar app: Azure Portal → App Service → OverView → Default Domain → click en URL publica

- c) En la app publicada realizar cambios en archivos
d) Verificar que los cambios se representen en storage account: Azure Portal → Storage Account → Storage Browser → Blob containers → Seleccionar su container → Verificar cambios



11. Preguntas de analisis

1. ¿Para qué sirve el archivo .env en la aplicación?
R/ Guarda variables de configuración, como endpoints y nombres de recursos, sin ponerlos directamente en el código.
2. ¿Por qué utilizar Blob Storage y no guardar los archivos dentro del proyecto?
R/ Porque permite almacenar archivos de forma externa, escalable y segura, sin cargar el proyecto.
3. ¿Cuál es la diferencia entre el endpoint de Blob Storage y la URL de App Service?
R/ El endpoint de Blob Storage permite acceder al almacenamiento de archivos; la URL de App Service permite acceder a la aplicación web.
4. ¿Por qué utilizar una identidad administrada en lugar de una Storage Account Key?
R/ Porque permite autenticar la aplicación con Azure sin guardar ni manejar claves de acceso.
5. ¿Qué función cumplen GitHub, Azure DevOps y la Service Connection?
R/GitHub almacena el código, Azure DevOps ejecuta el pipeline CI/CD y la Service Connection permite que Azure DevOps se conecte con Azure.
6. ¿Qué ocurre después de hacer push a main?
R/ Se activa el pipeline, que compila, publica y despliega la aplicación en Azure App Service.



7. Si el pipeline termina correctamente pero la aplicación recibe un 403 al subir un archivo, ¿qué revisarías?
R/ Revisaría los permisos de la identidad administrada, especialmente que tenga el rol Storage Blob Data Contributor sobre el Storage Account.

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-14/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.